Variable-Height Model Predictive Control using the VH-IP model

A.A. 2025 - 2026

Luca Franzin
Andrea Gravili
Giuseppe D'Addario
Federico Tranzocchi
Sapienza University of Rome

Contents

1 Introduction **2**
1.1 Background . . . 2
1.2 Objective . . . 3
1.3 Structure . . . 3

2 System modeling **4**
2.1 Dynamic balance on non-level ground . . . 4
2.2 VH-IP model . . . 4
2.2.1 Dynamic decoupling . . . 5
2.2.2 Preview and Control Horizon . . . 6

3 Control design **7**
3.1 Footstep planner . . . 7
3.2 IS-MPC . . . 8
3.2.1 Moving ZMP constraint . . . 9
3.2.2 QP-z . . . 9
3.2.3 QP-xy . . . 10

4 Simulation and results **14**
4.1 Software architecture . . . 14
4.2 State estimation . . . 14
4.3 Parameters . . . 14
4.4 Test trajectories . . . 15
4.5 Evaluation metrics . . . 15
4.6 Results . . . 16
4.6.1 Obstacle avoidance and box climbing . . . 16
4.6.2 Stair climbing and descent . . . 16
4.6.3 Discussion . . . 17

5 Conclusion **18**
5.1 Future work . . . 18

Chapter 1

Introduction

The deployment of humanoid robots in complex unstructured environments relies heavily on their ability to perform dynamic and versatile locomotion. Although walking on flat terrain has been successfully addressed in recent decades, navigating 3D environments (e.g., climbing stairs, crouching under obstacles) remains a highly challenging problem. This project focuses on the generation of dynamically stable 3D walking trajectories for the underactuated humanoid robot *HRP-4* in Fig. 1.1, using a *Model Predictive Control* (MPC) approach based on the framework proposed by Smaldone et al. [1].

Figure 1.1: The HRP-4 humanoid robot. Source: Kawada Industries/AIST.

1.1 Background

The most common approach to humanoid gait generation is based on the *Linear Inverted Pendulum* (LIP) model, which simplifies the robot's complex multibody dynamics by assuming a constant center of mass (CoM) height, and zero rate of change of the robot's angular momentum. Under this assumption, the dynamics of the *Zero Moment Point* (ZMP) (the point on the ground where the net tipping moments are zero) become linear, allowing for efficient real-time control

through *Quadratic Programming* (QP) optimization. However, the constant height assumption limits the robot's maneuverability, and consequently its adaptability to complex terrains like non-level ground.

Removing the constant height assumption yields the *Variable-Height Inverted Pendulum* (VH-IP) model, which accurately captures 3D locomotion, although introducing a nonlinearity: the varying height directly alters the natural frequency of the pendulum, which couples the vertical and horizontal dynamics. It is well known that nonlinear optimization is computationally expensive and too complex for real-time applications.

To overcome these limitations, we adopt an *Intrinsically Stable Model Predictive Control* (IS-MPC) architecture. By exploiting the triangular structure of the VH-IP dynamics, the control problem can be decoupled into two sequential QP problems. The first stage solves the vertical dynamics by treating the vertical ground reaction force (GRF) as a decision variable, allowing to define a time-varying natural frequency for the second stage, which then optimizes the horizontal CoM trajectory and footstep placements as a linear time-varying system.

1.2 Objective

The main goal of this project is to extend an existing IS-MPC framework [2] (that was originally designed for flat-ground walking) to support variable-height CoM dynamics, and validate it on the HRP-4 humanoid robot. Specifically, we aim to implement the two-stage QP formulation for real-time footstep adaptation and 3D CoM trajectory generation. Furthermore, since the natural frequency of the system becomes time-varying under the VH-IP model, we adapt the existing state estimator by designing a time-varying Kalman filter. The generated high-level references are then fed to an existing inverse dynamics-based whole-body controller. Thanks to this, we can carry out dynamic simulations to evaluate the overall performance and feasibility of the extended architecture.

1.3 Structure

The report is organized as follows. In Chapter 2, we present the system modeling, defining the VH-IP equations of motion and the dynamic decoupling strategy. In Chapter 3, we detail the control architecture, describing the high-level footstep planner and the specific formulation of the vertical and horizontal QPs. In Chapter 4, we describe the integration with the simulation environment and we present the results of our simulations for variable-height walking with different setups. Finally, in Chapter 5, we summarize our findings and discuss potential future work.

Chapter 2

System modeling

This chapter details the dynamic modeling of the humanoid robot used for 3D gait generation. We first introduce the concept of dynamic balance on non-level ground, and subsequently derive the equations of motion for the Variable-Height Inverted Pendulum (VH-IP) model, highlighting the decoupling of its vertical and horizontal components.

2.1 Dynamic balance on non-level ground

The fundamental requirement of any gait generation scheme is the ability to maintain dynamic balance. On flat ground, this is typically guaranteed by the ZMP criterion, which states that the ZMP must remain within the support polygon (the convex hull of the contact surfaces).

However, in 3D locomotion, contact surfaces are not necessarily coplanar, meaning a traditional 2D support polygon cannot be defined; instead, the admissible ZMP region $\mathcal{Z}$ for the VH-IP model is defined as

$$\mathcal{Z} = \left\{ \boldsymbol{p} : \boldsymbol{p} = \sum_{j=1}^{N_c} \gamma_j \boldsymbol{v}_j, \text{ with } \gamma_j \geq 0, \sum_{j=1}^{N_c} \gamma_j = 1 \right\} \tag{2.1}$$

where $\boldsymbol{p}$ is a generic 3D point and $\boldsymbol{v}_j$, for $j = 1, \ldots, N_c$, are the vertexes of all active contact surfaces.

In a world of stairs, where all contact surfaces are parallel to the ground, this region takes the shape of an oblique prism during double support phases (Fig. 2.1). On flat ground, $\mathcal{Z}$ simply reduces to the standard support polygon. To maintain linearity in the control formulation, the ZMP is constrained to a fixed-shape region (the footprint) that horizontally slides between consecutive footsteps during double support, effectively acting like a horizontal slice of the prism.

2.2 VH-IP model

To characterize the relationship between the CoM and the ZMP while allowing for height variations, we use the VH-IP model. Assuming a zero rate of change of the angular momentum around the CoM, the Newton-Euler equations for a humanoid are given by

$$m(\ddot{\boldsymbol{p}}_c - \boldsymbol{g}) = \boldsymbol{f} \tag{2.2}$$

$$(\boldsymbol{p}_z - \boldsymbol{p}_c) \times \boldsymbol{f} = \boldsymbol{0} \tag{2.3}$$

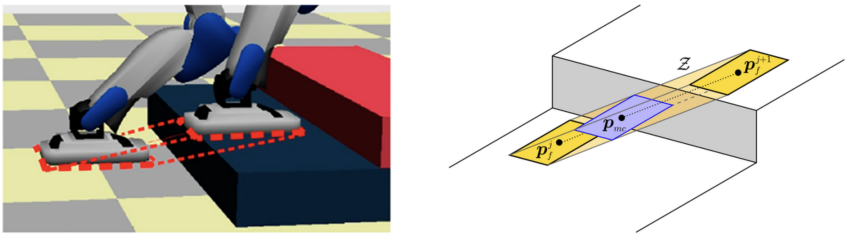


Figure 2.1: Visual representation of the admissible ZMP region $\mathcal{Z}$ (yellow) during a double support phase on non-level terrain. The moving constraint region (blue) represents a horizontal slice of the oblique prism that dynamically slides between consecutive footsteps.

where m is the total mass of the robot, $\boldsymbol{p}_c = (x_c, y_c, z_c)$ is the CoM position, $\boldsymbol{g} = (0, 0, -g)$ is the gravity acceleration vector, $\boldsymbol{f}$ is the GRF, and $\boldsymbol{p}_z = (x_z, y_z, z_z)$ is the ZMP position. A schematic representation of these variables is shown in Fig. 2.2.

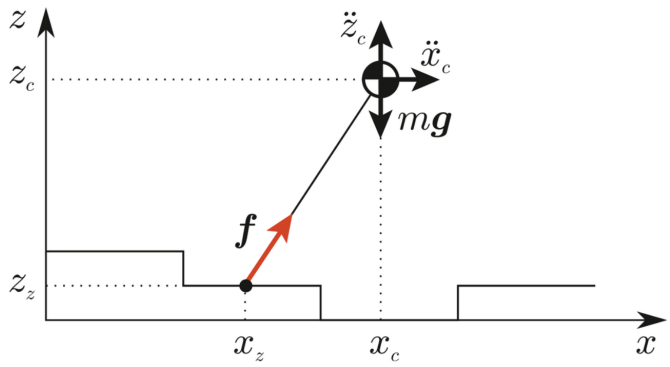


Figure 2.2: VH-IP model in the x-z plane. We can see the relationship between the CoM ($\boldsymbol{p}_c$), the ZMP ($\boldsymbol{p}_z$), and the GRF ($\boldsymbol{f}$).

2.2.1 Dynamic decoupling

The core advantage of the VH-IP model lies in its triangular structure, which allows us to decouple the vertical and horizontal dynamics. By expanding (2.2) and (2.3), the vertical CoM dynamics can be directly extracted from the third component of the Newton equation:

$$\ddot{z}_c = \frac{f_z}{m} - g \tag{2.4}$$

where f_z is the vertical component of the GRF. The horizontal dynamics are then obtained as:

$$\ddot{x}_c = \lambda(x_c - x_z) \tag{2.5}$$

$$\ddot{y}_c = \lambda(y_c - y_z) \tag{2.6}$$

where λ represents the (time-varying) squared natural frequency of the pendulum:

$$\lambda = \frac{\ddot{z}_c + g}{z_c - z_z} = \frac{f_z}{m(z_c - z_z)} \tag{2.7}$$

With this formulation, the vertical dynamics (2.4) are completely independent of the horizontal dynamics (2.5)-(2.6). Once a profile for the vertical GRF f_z is chosen, the vertical CoM acceleration $\ddot{z}_c$ is strictly determined, which in turn defines the evolution of λ. Consequently, the horizontal dynamics can be treated as a linear time-varying system. This structural decoupling is the foundation upon which the two-stage MPC architecture in Chapter 3 is built.

2.2.2 Preview and Control Horizon

The MPC operates in discrete time with a sampling interval δ. To generate a stable and computationally feasible trajectory, we must distinguish between two different time horizons: the *preview horizon* and the *control horizon* (see Fig. 2.3).

The preview horizon, denoted by $T_p = P\delta$ (where P is the number of preview steps), defines the total look-ahead window. We use it to incorporate future information from the footstep plan, such as upcoming target positions and height references. Using a sufficiently long preview horizon guarantees that the robot's current state is compatible with the future requirements, and prevents it from entering states from which the next footstep would be kinematically or dynamically unreachable.

The control horizon, denoted by $T_c = C\delta$ (where C is the number of control steps), is the window over which the MPC actually optimizes the decision variables (e.g., the ZMP and the vertical GRF). In practice, T_c is chosen such that $T_c \leq T_p$.

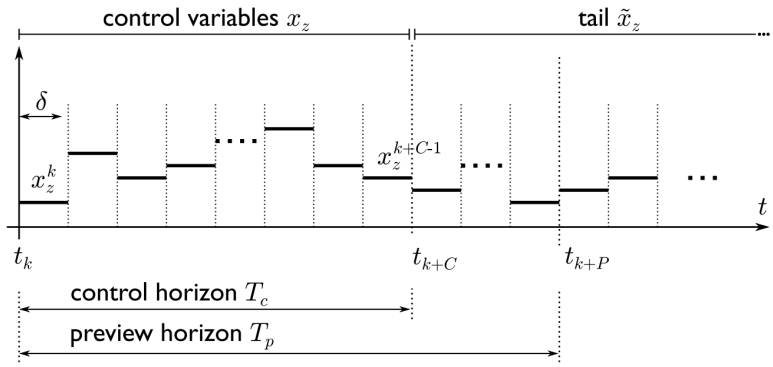


Figure 2.3: Control and preview horizon.

Chapter 3

Control design

This chapter details the design of the control system used to generate dynamically stable 3D walking trajectories. We begin by describing the high-level footstep planner responsible for providing the reference inputs to the controller. We then focus on the two-stage *Intrinsically Stable Model Predictive Control* (IS-MPC) architecture, detailing the formulation of both the vertical (QP-z) and horizontal (QP-xy) quadratic programming problems, as illustrated in Fig. 3.1.

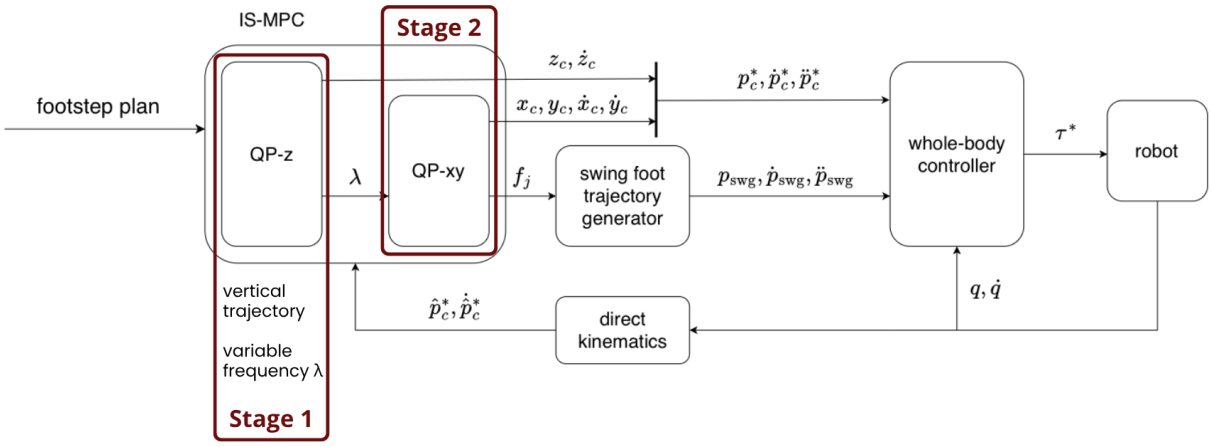


Figure 3.1: Block scheme of the proposed control architecture. The high-level footstep plan is processed by the two-stage IS-MPC (QP-z and QP-xy) to generate the desired CoM and ZMP trajectories, which are then tracked by a whole-body controller.

3.1 Footstep planner

The footstep planner is the high-level component of our control architecture. Its function is to translate a sequence of high-level commands into a plan that the IS-MPC can use to generate stable trajectories.

For every step in the desired trajectory, the planner requires a set of reference parameters. We use these inputs to specify the intended movement for that specific step:

- the linear desired velocities $\dot{x}, \dot{y}$ of the robot's center in the sagittal and coronal directions;
- the desired rate of change $\dot{\theta}$ of the robot's heading;

- the change in the ground height Δz between the current step and the next;
- the desired distance h_{ref} between the CoM and the ZMP;
- the boundaries of a planar region defining the walkable area at the current elevation.

To determine the target footstep positions, the planner uses a virtual unicycle model: it simulates the evolution of the robot's center in the global frame by integrating the reference velocities over the duration of each step. For each step, the planner updates the unicycle's orientation θ and position $\boldsymbol{p}_{\text{uni}}$ using the provided velocities:

$$\theta_{k+1} = \theta_k + \dot{\theta}\delta \tag{3.1}$$

$$\boldsymbol{p}_{\text{uni},k+1} = \boldsymbol{p}_{\text{uni},k} + \boldsymbol{R}(\theta_{k+1}) \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \delta \tag{3.2}$$

where $\boldsymbol{R}(\theta)$ is the 2D rotation matrix and δ is the time step.

The actual target footstep position $\boldsymbol{p}_{\text{foot}}$ is derived by applying a lateral offset to the center of the unicycle. This offset is computed based on the current heading θ to place the feet symmetrically relative to the body's centerline. The vertical coordinate of the foot is determined by the accumulation of the vertical displacements $\sum \Delta z$, allowing the planner to generate paths over non-level terrain, such as stairs.

In addition to the geometry, the planner defines the timing of the gait. Each step is divided into a *single support* (SS) phase and a *double support* (DS) phase. The duration of these phases is predefined and stored in the final plan to synchronize the ZMP constraints with the actual contact state of the robot.

The final output of the planner is a sequence of steps, where for each step, the plan stores the following information:

- the target 3D position $\boldsymbol{p}_{\text{foot}}$ and orientation of the foot;
- the durations of the SS and DS phases;
- the identifier of the support foot;
- the reference CoM height h_{ref} for that step.

The MPC controller queries this plan in real-time using the simulation time t to determine the active step index and the current gait phase, ensuring that the generated trajectories are synchronized with the intended footstep sequence.

3.2 IS-MPC

The goal of the IS-MPC is to generate a dynamically stable 3D CoM trajectory and adapt the planned footsteps to ensure the robot remains balanced. Following the triangular structure of the VH-IP dynamics described in Chapter 2, the control problem is decomposed into two sequential stages: a vertical optimization (QP-z) and a horizontal optimization (QP-xy).

At each time step k, the controller solves two QP problems over a control horizon T_c. The first stage (QP-z) determines the vertical GRF to track the desired CoM height. This result is then used to compute the time-varying natural frequency λ, which serves as a parameter for the second stage (QP-xy). The second stage optimizes the horizontal ZMP and the actual footstep locations to maintain balance and follow the global plan.

3.2.1 Moving ZMP constraint

We said the robot is dynamically balanced if the ZMP is inside $\mathcal{Z}$. Expressing this constraint in DS would involve products of decision variables, yielding a nonlinear constraint. We preserve linearity by employing a *moving constraint*: the ZMP must belong to a region (the footprint) which slides between consecutive footsteps during DS and coincides with a footstep during SS. This region is basically a horizontal slice of the oblique prism. The center of the moving constraint region is denoted by $\boldsymbol{p}_{mc} = (x_{mc}, y_{mc}, z_{mc})^T$ and is expressed as a linear function of the footstep positions:

$$\boldsymbol{p}_{mc}(t) = \sum_{j=0}^{F} \alpha^j(t) \boldsymbol{p}_f^j \tag{3.3}$$

where $\alpha^j(t)$ are piecewise-linear functions of time in $[0, 1]$ and such that

$$\sum_{j=0}^{F} \alpha^j(t) = 1$$

and $\boldsymbol{p}_f^j = (x_f^j, y_f^j, z_f^j)^T$ denotes the position of the j-th footstep. Since the moving constraint region is planar, the vertical component z_z of the ZMP must at all times coincide with z_{mc}:

$$z_z(t) = z_{mc}(t) \tag{3.4}$$

3.2.2 QP-z

The goal of QP-z is to generate the vertical CoM trajectory z_c. The prediction model is based on 2.4, where the decision variable is the vertical GRF f_z, assumed to be piecewise constant over the control horizon:

$$\ddot{z}_c = \frac{f_z^{k+i}}{m} - g \quad \text{for } t \in [t_{k+i}, t_{k+i+1}], \quad i = 0, \ldots, C-1 \tag{3.5}$$

During the walking motion, we guarantee that the robot doesn't slip by enforcing a simplified constraint:

$$f_z^{k+i} \geq f_z^{\min} \tag{3.6}$$

Here, $f_z^{\min}$ is a lower bound to f_z that provides enough friction to avoid slipping.

We collect all decision variables in a vector

$$\boldsymbol{F}_z^k = (f_z^{k+1}, \ldots, f_z^{k+C-1})^T$$

along with the predicted state variables

$$\dot{\boldsymbol{Z}}_c^k = \left(\dot{z}_c^k, \ldots, \dot{z}_c^{k+C-1}\right)^T \quad \boldsymbol{Z}_c^k = \left(z_c^k, \ldots, z_c^{k+C-1}\right)^T$$

and the absolute reference values for the CoM height

$$\boldsymbol{Z}_c^{k,*} = \left(z_z^k + h^k, \ldots, z_z^{k+C-1} + h^{k+C-1}\right)^T$$

In the latter, z_z^{k+i} is the vertical coordinate z_{mc} of the center of the planar moving constraint region, while h^{k+i} is the reference CoM height relative to the ZMP.

QP-z is formulated as

$$\begin{aligned} \min_{p_z^k} \quad & \|Z_c^k - Z_c^{k,*}\|^2 + \alpha_z \|\dot{Z}_c^k\|^2 + \beta_z \|\Delta F_z^K\|^2 \\ \text{s.t.} \quad & f_z^{k+i} \geq f_z^{\min} \end{aligned} \tag{3.7}$$

where α_z, β_z are positive weights. The first term of the cost function tracks the reference CoM height, while the second term penalizes sudden height variations; the last term reduces the difference between consecutive samples of f_z, so as to generate a smoother GRF profile. Once solved, we can use the resulting f_z to compute $(z_c(t), \dot{z}_c(t))$.

3.2.3 QP-xy

The goal of QP-xy is to complete the generation of the CoM trajectory with the horizontal components. With λ_k being computed using the CoM trajectory generated by QP-z, the horizontal dynamics can be expressed as (piecewise-time-invariant):

$$\begin{aligned} \ddot{x}_c^k &= \lambda^{k+i}(x_c - x_z) \\ \ddot{y}_c^k &= \lambda^{k+i}(y_c - y_z) \end{aligned} \quad \text{for } t \in [t_{k+i}, t_{k+i+1}), \quad i = 0, \ldots, C-1 \tag{3.8}$$

with:

$$\lambda^{k+i} = \frac{\ddot{z}_c^{k+i} + g}{z_c^{k+i} - z_z^{k+i}} \tag{3.9}$$

We enforce five kinds of constraints: ZMP, ground patch, kinematic, swing foot, and stability. All the planned footsteps have the same orientation (x-axis), so we can decouple the motion along the sagittal and coronal plane. Thanks to this, we can describe the constraints for the x-component only since those for the y-component would be the same.

ZMP constraint

To guarantee dynamic balance, the ZMP must be confined to the moving constraint region, which is a rectangular area of dimensions $d_{z,x}$ and $d_{z,y}$ centered at (x_{mc}, y_{mc}):

$$-\frac{1}{2} d_{z,x} \leq x_z^k - x_{mc}^k \leq \frac{1}{2} d_{z,x} \tag{3.10}$$

Ground patch constraint

To ensure the adapted footsteps land on valid surfaces and do not slide off the edges of platforms or stairs, each adapted footstep (x_f^j, y_f^j) must remain within the convex ground patch $\mathcal{P}_{\text{conv}}$ associated with the candidate footstep. These patches are predefined based on the known elevations of the objects in the simulation environment.

To guarantee that the entire physical area of the foot is supported by the patch, and not just its center point, the constraint is enforced using a Minkowski difference:

$$\left(x_f^j, y_f^j\right) \in \mathcal{P}_{conv} \left(\hat{x}_f^j, \hat{y}_f^j\right) \ominus \mathcal{P}_{foot} \tag{3.11}$$

where $\mathcal{P}_{\text{foot}}$ is a rectangular region representing the dimensions of the robot's foot. This effectively shrinks the admissible ground patch by the size of the foot, ensuring that the foot is always fully contained within the valid surface. A visual representation of these convex regions and the resulting constrained areas is shown in Fig. 3.2.

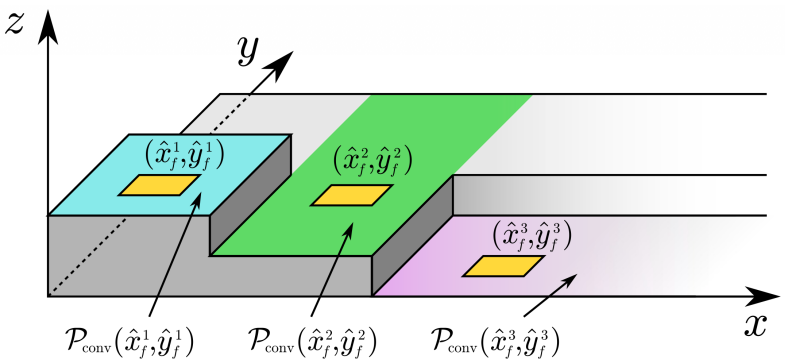


Figure 3.2: Convex ground patches $\mathcal{P}_{conv}$ used to constrain the adapted footstep positions. The colored regions represent the valid surfaces extracted from the environment elevation map.

Kinematic constraint

The reachable workspace of the robot's legs imposes limits on the maximum step length. These limits are modeled as a box constraint that dynamically shrinks as the vertical height difference between consecutive steps increases. For the x-axis, this is expressed as

$$|x_f^j - x_f^{j-1}| \leq \frac{1}{2} d_{a,x}(z_f^j, z_f^{j-1}) \tag{3.12}$$

The admissible region $d_{a,x}$ is computed using a scaling factor:

$$d_{a,x} = \left(1 - \sigma \frac{|z_f^j - z_f^{j-1}|}{dz_{max}}\right) d_{a,x}^0 \tag{3.13}$$

where $d_{a,x}^0$ is the maximum step length on flat ground, dz_{max} is the maximum feasible height variation, and $\sigma \in (0, 1)$ is a tuning parameter.

An analogous constraint is applied to the y-axis: however, it differs from the x-expression to account for the robot's lateral stance. Since the feet alternate positions across the sagittal plane, the y-constraint is centered around a lateral offset ℓ, ensuring that the adapted footsteps remain within a kinematically reachable range relative to the opposite foot.

Swing foot constraint

To prevent the leg joint speeds from exceeding a feasible range during the swing motion, the distance to the next footstep placement is limited by the remaining time in the current SS phase. As the landing time approaches, the admissible reach of the swing foot dynamically shrinks, as shown in Fig. 3.3. This is expressed as:

$$|x_f^j - x_{sw}^k| \leq (t_s^0 + T_{ss}^0 - t_k) v_{sw}^{max} \tag{3.14}$$

where x_{sw}^k is the current swing foot position, t_s^0 is the start time of the current step, T_{ss}^0 is the SS duration, and v_{sw}^{max} is the maximum achievable Cartesian velocity of the foot.

Stability constraint

Keeping the ZMP inside the support polygon is a *necessary but not sufficient* condition for balance. In fact, even if the ZMP is perfectly centered, the CoM can still gain too much momentum and drift away from the ZMP, making the robot fall. To address this problem, we must

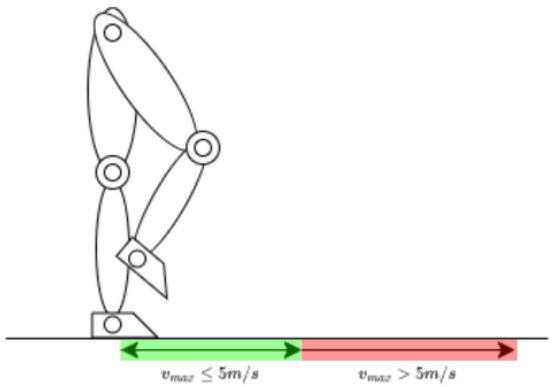


Figure 3.3: The maximum reachable distance to the next footstep decreases linearly as the remaining time in the single support phase decreases.

ensure that the CoM trajectory remains bounded with respect to the ZMP: this way, we force the current CoM position and velocity to be compatible with the future footsteps.

For the LTV system in (3.8), the stability condition is formulated by assuming that, after the control horizon, the dynamics revert to a time-invariant LIP system with a constant frequency $\lambda_{LIP} = g/(z_c^N + h_{ref}^N)$.

To make this condition computable within the QP, we express it as a linear equality constraint by splitting the trajectory into a known part over the control horizon and an "anticipative tail" for the remaining time, where the future ZMP is assumed to follow the center of the planned footsteps $\hat{x}_z(\tau)$. This results in the final linear equality constraint:

$$\boldsymbol{G} \sum_{i=0}^{C-1} \prod_{j=i}^{C-1} \boldsymbol{\Phi}_{k+j} \boldsymbol{B}_{k+i} x_z^{k+i} = \sqrt{\lambda_{\text{LIP}}} \int_{t_{k+C}}^{\infty} e^{-\sqrt{\lambda_{\text{LIP}}}(\tau - t_{k+C})} \tilde{x}_z(\tau) d\tau - \boldsymbol{G} \prod_{i=0}^{C-1} \boldsymbol{\Phi}_{k+i} \boldsymbol{x}_c^k \quad (3.15)$$

where $\boldsymbol{G} = \begin{bmatrix} 1 & 1/\sqrt{\lambda_{LIP}} \end{bmatrix}$, and $\boldsymbol{\Phi}_{k+j}$ and $\boldsymbol{B}_{k+i}$ are the time-varying state transition and input matrices of the discretized horizontal dynamics. The complete derivation of this constraint and the corresponding stability proof are provided in [1].

Full formulation

To formulate the problem as a quadratic program, we collect the decision variables and the associated parameters into vectors. The decision variables are the horizontal ZMP positions over the control horizon and the adapted footstep positions:

$$\boldsymbol{X}_z^K = (x_z^{k+1}, \ldots, x_z^{k+N})^T, \quad \boldsymbol{X}_f^k = (x_f^1, \ldots, x_f^F)^T \quad (3.16)$$

Along with these, we define the vector containing the x-coordinates of the center of the moving constraint region $\boldsymbol{X}_{\text{mc}}^k = (x_{\text{mc}}^{k+1}, \ldots, x_{\text{mc}}^{k+N})^T$, and the vector of candidate footstep positions provided by the planner $\hat{\boldsymbol{X}}_f^k = (\hat{x}_f^1, \ldots, \hat{x}_f^F)^T$. Finally, to penalize sudden variations in the ZMP, we define the ZMP increment vector as

$$\Delta \boldsymbol{X}_z = (x_z^{k+1} - x_z^k, \ldots, x_z^{k+N} - x_z^{k+N-1})^T \quad (3.17)$$

The complete QP-x problem is formulated as follows:

$$
\begin{aligned}
\min_{\boldsymbol{X}_z^k, \boldsymbol{X}_f^k} \quad & \|\boldsymbol{X}_z^k - \boldsymbol{X}_{mc}^k\|^2 + \alpha_x \|\Delta \boldsymbol{X}_z\|^2 + \beta_x \|\boldsymbol{X}_f^k - \hat{\boldsymbol{X}}_f^k\|^2 \\
\text{s.t.} \quad & -\frac{1}{2}\begin{pmatrix} d_{z,x} \\ d_{z,y} \end{pmatrix} \leq \begin{pmatrix} x_z^{k+i} \\ y_z^{k+i} \end{pmatrix} - \begin{pmatrix} x_{mc}^{k+i} \\ y_{mc}^{k+i} \end{pmatrix} \leq \frac{1}{2}\begin{pmatrix} d_{z,x} \\ d_{z,y} \end{pmatrix} \\
& \left(x_f^j, y_f^j\right) \in \mathcal{P}_{\text{conv}}\left(\hat{x}_f^j, \hat{y}_f^j\right) \ominus \mathcal{P}_{\text{foot}} \\
& \left|x_f^j - x_f^{j-1}\right| \leq \frac{d_{a,x}\left(z_f^{j-1}, z_f^j\right)}{2} \\
& \left|x_{\text{sw}}^k - x_f^1\right| \leq \left(t_s^0 + T_{ss}^0 - t_k\right) v_{\text{sw}}^{\max} \\
& \boldsymbol{G} \sum_{i=0}^{C-1} \prod_{j=i}^{C-1} \boldsymbol{\Phi}_{k+j} \boldsymbol{B}_{k+i} x_z^{k+i} = \sqrt{\lambda_{\text{LIP}}} \int_{t_{k+C}}^{\infty} e^{-\sqrt{\lambda_{\text{LIP}}}(\tau - t_{k+C})} \tilde{x}_z(\tau) d\tau - \boldsymbol{G} \prod_{i=0}^{C-1} \boldsymbol{\Phi}_{k+i} \boldsymbol{x}_c^k
\end{aligned}
\tag{3.18}
$$

where α_x, β_x are positive weights. The first term of the cost function minimizes the distance between the ZMP and the center of the moving constraint region, therefore maximizing the stability margin. The second term penalizes sudden variations in the ZMP to ensure a smooth trajectory. The last term minimizes the deviation of the adapted footsteps from the candidate positions provided by the planner.

Chapter 4

Simulation and results

In this chapter, we detail the practical implementation of the control architecture described in Chapter 3. Specifically, we present the software tools used to build the simulation environment, the parameters of the robot and the controllers, and explain the metrics used to evaluate the performance of our approach. Finally, we analyze the results based on the defined metrics specifically focusing on the tracking accuracy of the CoM and the ZMP.

4.1 Software architecture

The simulation framework is implemented entirely in Python 3, relying on different libraries to handle rigid body dynamics, optimization, and mathematical computations. DART is used as the simulation's engine, handling both the environment and the computation of kinematic and dynamic properties of the humanoid. CasADi and OSQP are used to solve the QP problem presented in the previous chapter.

4.2 State estimation

To close the control loop, the current state of the robot must be accurately estimated. Due to the nature of the VH-IP model where λ is time-varying, a standard Kalman filter is insufficient; therefore, we implement a time-varying version.

At each time step k, the state transition matrix $\boldsymbol{A}$ and the control matrix $\boldsymbol{B}$ are updated based on the current estimated height and the vertical GRF. The filter processes the simulation data to provide a filtered estimate of the CoM position and velocity, as well as the ZMP position. This estimated state $\boldsymbol{x}^k$ is then used as the initial condition for the two-state MPC optimization described in Chapter 3.

4.3 Parameters

The simulation environment operates at 100 Hz and uses these parameters: robot's mass $m = 43.0$ kg, nominal CoM height $h = 0.72$ m, foot dimensions = 0.25×0.13 [m]. The gait is defined by these parameters: SS duration = 0.7 s, DS duration = 0.3 s, maximum swing foot velocity $v_{\text{sw}}^{\text{max}} = 2.0$ m/s, control horizon $C = 100$ steps, and preview horizon $P = 200$ steps. The weights of the cost function for horizontal optimization are: CoM velocity weight $\alpha_{xy} = 1$, footstep weight $\beta_{xy} = 10$. For vertical optimization: CoM velocity weight $\alpha_z = 1$, ground force variation weight $\beta_z = 10$.

4.4 Test trajectories

We designed two trajectories to test the correctness of the proposed system implementation, both requiring the robot to change it's CoM height in accordance to different elements in the environment.

Obstacle and box

The reference sequence in this trajectory, shown in Fig 4.1, commands the robot to perform some forward steps while lowering its CoM height in order to avoid an obstacle by crouching down. After this phase, the robot is commanded to climb onto a 10 cm tall box. Once up, it is requested to continue walking forward, changing its CoM height, up to the descent phase. Finally, the robot performs additional steps on the ground before the simulation stops.

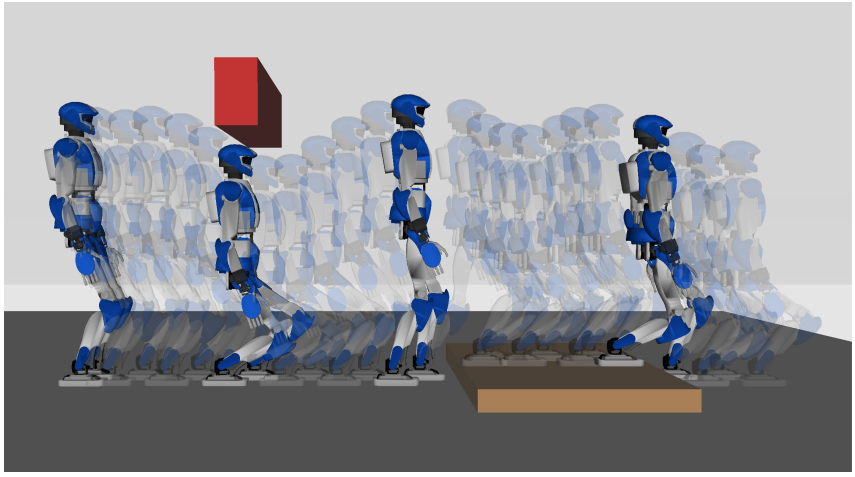


Figure 4.1: Stroboscopic view of the trajectory

Stairs

For this trajectory, shown in Fig 4.2, the task assigned to the robot is to climb and descend the stairs. Each step is 15 cm tall. Between the ascent and descent phases there is a small plateau in which the robot is requested to walk straight lowering its CoM height in order to prepare itself for the descent phase. The simulation stops after a few steps on the ground.

4.5 Evaluation metrics

To validate the effectiveness of the proposed framework, we measured three quantities.

CoM tracking error The deviation between the CoM reference trajectory generated by the IS-MPC (continuous line) and the actual CoM position executed by the WBC (dashed line). This is evaluated along both three axes (x, y, and z) to verify both horizontal and vertical tracking.

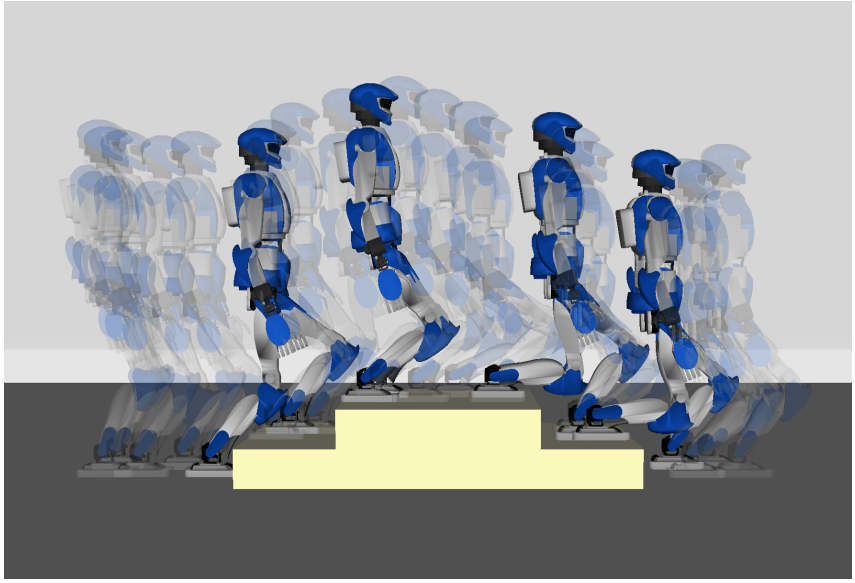


Figure 4.2: Stroboscopic view of the trajectory

ZMP tracking error The deviation between the measured ZMP (dashed line), which is computed using the ground reaction forces and the contact points, and the reference ZMP (continuous line) computed by the controller. Even this quantity is evaluated along the three axes to verify both horizontal and vertical tracking. The latter is important when there are stairs or boxes that cause contact to happen at a different height from the 0-level ground.

4.6 Results

4.6.1 Obstacle avoidance and box climbing

The first scenario tests the robot's ability to modulate its CoM height dynamically to traverse a complex environment.

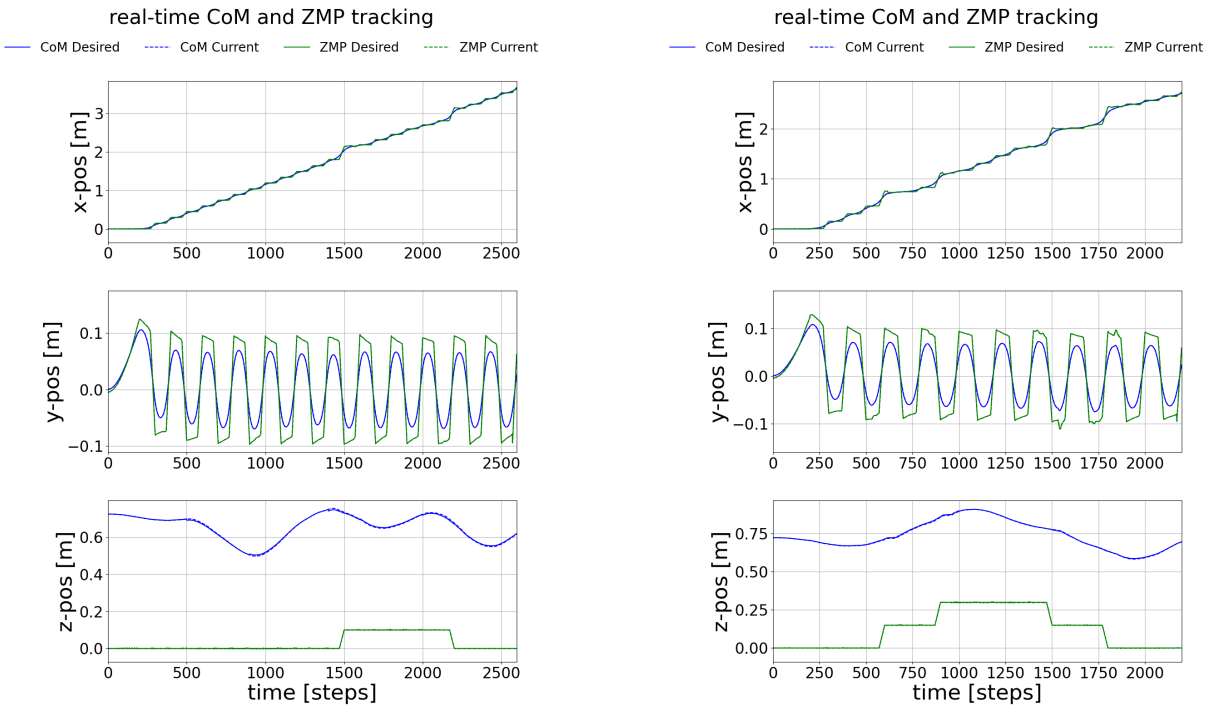
The resulting CoM and ZMP trajectories are shown in Fig. 4.3a. In the sagittal plane (x-pos), the CoM tracking is nearly perfect, with the actual trajectory closely following the reference. In the coronal plane (y-pos), the characteristic oscillatory motion of the walking gait is maintained, and the tracking error remains minimal throughout the entire sequence.

The vertical profile (z-pos) provides the most significant validation of the VH-IP model. The robot successfully reduces its CoM height to avoid the obstacle and then increases it to climb the box. The tracking error in z is remarkably low, demonstrating that the QP-z stage effectively realizes the desired height variations. Furthermore, the ZMP tracking profiles show that the vertical component of the ZMP correctly shifts when the robot steps onto the box, confirming that the moving constraint region is correctly adapted to non-level terrain.

4.6.2 Stair climbing and descent

This task is particularly challenging as it requires frequent and discrete changes in the support surface height, which can easily destabilize a robot if the CoM momentum is not properly managed.

The tracking profiles in Fig. 4.3b show that the CoM maintains a stable trajectory despite the discontinuous changes in the ground elevation. The vertical ZMP trajectory follows a step-like



(a) Trajectory 1 final plot.

(b) Trajectory 2 final plot

Figure 4.3: CoM and ZMP tracking profiles.

profile, precisely matching the height of each stair.

The high fidelity of the ZMP tracking indicates that the horizontal optimization (QP-xy) is successfully adapting the footstep positions to keep the ZMP within the admissible region $\mathcal{Z}$, even when the support surfaces are not coplanar. The absence of significant divergences in the CoM trajectory validates the effectiveness of the stability constraint implemented in the IS-MPC, ensuring that the robot does not gain excessive momentum during the descent phase.

4.6.3 Discussion

Comparing the results of both trajectories, we observe that the two-stage QP architecture allows for a seamless integration of vertical and horizontal balance. The decoupling of the dynamics enables the robot to handle significant height variations without compromising the stability of the horizontal gait.

The tracking errors for both CoM and ZMP remain within acceptable bounds, proving that the VH-IP model is a sufficient abstraction for these tasks. Our results demonstrate that the variable-height extension is robust for slow-to-medium speed navigation in ”worlds of stairs” and environments requiring vertical agility.

Chapter 5

Conclusion

We presented a gait generation algorithm that extends the basic IS-MPC to a 3D case, in order to achieve a full walking system, by implementing a Variable Height Inverted Pendulum (VH-IP), which allows to walk on stairs, moving under and also over obstacles. We simulated two different trajectories executed by the HRP-4 humanoid robot and measured the performance of the system.

The results of our experiments show that the decoupling of the horizontal and vertical dynamics allows us to command a variation in the robot CoM height without sacrificing computational time, yielding to a very efficient MPC implementation. Furthermore, as can be seen from both plots in Fig. 4.3a and 4.3b, the tracking error for both the CoM and ZMP trajectory are extremely low, proving the capability of the proposed MPC strategy to achieve accurate trajectory tracking while guaranteeing stable locomotion behavior.

Finally, it's important to note that the optimization of the footsteps performed by the MPC is strictly dependent on the ground patches defined exploiting the full knowledge of the environment. This is a strong limitation because it implies that adding or modifying a scenario requires to re-define these patches.

5.1 Future work

This work can be expanded in several directions, with the aim of enhancing the robustness of humanoid locomotion while also adding new features. Some possible developments include:

- extend the robot's choice of movements, by implementing, for example, a flight phase, allowing the robot to run or move on tilted surfaces, as defined in [1];
- add the use of Reinforcement Learning methods in the pipeline to implement push recovery capability, without adding too much computational load and complicating the MPC;
- equip the robot with sensors that provide visual feedback. Using external measures would remove the need to have full knowledge of the environment, allowing on-line ground patches generation and thus making the system reliable also in unknown environments.

These are just some of the possible extensions, as the complexity of such systems allows for many further implementations.

Bibliography

[1] F. M. Smaldone, N. Scianca, L. Lanari, and G. Oriolo, "From walking to running: 3d humanoid gait generation via mpc," *Frontiers in Robotics and AI*, vol. 9, p. 876 613, 2022.

[2] DIAG Robotics Lab, *Intrinsically Stable Model Predictive Control (IS-MPC)*, GitHub repository. [Online]. Available: https://github.com/DIAG-Robotics-Lab/ismpc.